

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 01 · LECTURE NOTES

What Analysis Means

Models of computation, growth, and the honest limits of asymptotic reasoning.

MODULE OUTCOMES

- State what a model of computation is and why analysis requires one.
- Use O , Ω , and Θ precisely, including what each does not say.
- Distinguish worst-case, average-case, and amortised analysis.
- Explain where asymptotic analysis stops predicting real running time.
- Build a benchmarking harness and read a log-log plot.

1 What are we counting?

The question 'how fast is this algorithm?' has no answer until you say what machine it runs on, and real machines are far too complicated to prove anything about. So we analyse algorithms against a **model of computation**: an idealised machine simple enough to reason about and close enough to reality to be useful.

The default is the **RAM model**. It assumes a processor executing instructions one at a time, with arithmetic, comparison, and memory access on word-sized values each costing one unit of time. Memory is uniform — every access costs the same, regardless of address or history.

THAT LAST ASSUMPTION IS FALSE

On real hardware, an L1 cache hit costs about 4 cycles and a main-memory miss about 200. Memory access is not uniform; it is not close to uniform.

This means the RAM model *cannot distinguish* an algorithm with good memory locality from one without. Two algorithms with identical Θ bounds can differ by an order of magnitude on a real machine, and the model is blind to it.

This is not a reason to abandon the model. It is a reason to know what it does not see. CSCE 614 runs alongside this course precisely so you learn both halves at once.

1.1 Other models, and what each is for

Model	One step is	Lets you prove
RAM	An arithmetic op or a memory access	Nearly everything in this course. The default.

Model	One step is	Lets you prove
Comparison model	A comparison between two elements	The $\Omega(n \log n)$ sorting lower bound, which is a statement about <i>all possible</i> comparison sorts.
External memory	A block transfer between disk and memory	Why B-trees have the branching factor they do.
Cell probe	A read or write of one memory cell	Data-structure lower bounds, independent of computation cost.
PRAM / work-span	A parallel step across many processors	Parallel speedup limits. CSCE 735 uses this.

Choosing a model is choosing which resource you think is scarce. That is a modelling judgement, and getting it wrong is how you end up optimising something that was never the bottleneck.

2 Asymptotic notation, precisely

The definitions are worth writing out in full, because almost every common misuse comes from forgetting a quantifier.

$$\begin{aligned}
 f = O(g) &\Leftrightarrow \exists c > 0, n_0 : f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0 \\
 f = \Omega(g) &\Leftrightarrow \exists c > 0, n_0 : f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0 \\
 f = \Theta(g) &\Leftrightarrow f = O(g) \text{ and } f = \Omega(g)
 \end{aligned}$$

2.1 Four things the notation does not say

- O is not a description.** It is an upper bound, and upper bounds compose upward freely. Binary search is $O(n^3)$. The statement is true and conveys nothing. When you know the tight bound, say Θ .
- Constants are invisible, and constants decide programs.** An $O(n)$ algorithm with a constant of 10^6 loses to an $O(n^2)$ algorithm until n exceeds a million. Galactic algorithms — asymptotically optimal, never used — are an entire genre.
- Nothing is claimed below n_0 .** This is not a technicality: most real inputs are small. Production sort implementations switch to insertion sort below about 16 elements for exactly this reason.
- Worst case is a guarantee, not a prediction.** Quicksort's worst case is $\Theta(n^2)$; it is the standard sort anyway, because the worst case is vanishingly unlikely once randomised.

3 Which case are you analysing?

Kind	Averages over	Strength
Worst case	Nothing — it is a maximum over all inputs	A guarantee. Holds always, including under attack.

Kind	Averages over	Strength
Average case	An assumed distribution on inputs	Only as good as the assumption. Real data is rarely uniformly random — sorted, nearly sorted, and heavily duplicated inputs are all common.
Expected case (randomised)	The algorithm's own coin flips	Holds for <i>every</i> input. Strictly stronger than average case, and the reason randomisation is so useful.
Amortised	A sequence of operations	A guarantee on the total, not on any single operation. Different from average case: no probability is involved at all.

THE DISTINCTION THAT MATTERS MOST

Average case says: if your input is drawn from distribution D , the expected cost is X . An adversary who knows your algorithm can simply supply an input outside D .

Expected case for a randomised algorithm says: for *any* input whatsoever, the expected cost over my internal randomness is X . The adversary cannot do anything about this, because they cannot see your coins. This is precisely why randomised quicksort replaced deterministic quicksort. The deterministic version has a worst case an attacker can trigger; the randomised version has one that no attacker can target. Module 03 makes this rigorous.

Amortised analysis is a separate idea that students routinely conflate with average case. It involves no probability at all. It is a worst-case statement about a *sequence*: the total cost of n operations is bounded, even though individual operations may be expensive. Module 05 covers it properly.

4 Where the model stops predicting

This section is normally omitted from algorithms courses, which is a mistake. The gap between analysis and measurement is not noise; it is structured, explicable, and worth understanding.

Effect	Why the model misses it	Typical magnitude
Memory hierarchy	The RAM model charges 1 for every access regardless of locality.	10–50×. Sequential array traversal versus pointer chasing through a linked list is the canonical example, and the asymptotics are identical.
Branch misprediction	Control flow is free in the model.	~15 cycles per miss. A branchless binary search can beat a branchy one substantially despite doing more work.
Constant factors	Deliberately discarded by Θ .	Decisive below n_0 . Real sorts hybridise for this reason.
Allocation	Memory is free and infinite in the model.	An allocation costs far more than an arithmetic operation, and allocating per element can dominate everything else.
Vectorisation	No notion of parallel execution within one step.	8–16× on predictable loops, which is why a linear scan can beat a binary search on small arrays.

The method this course adopts: **prove in the model, measure on the machine, and explain every gap**. A gap you cannot explain is a sign that you have misunderstood either the algorithm or the machine, and either way it is worth finding out which.

5 Measuring properly

```
import time, statistics

def bench(fn, make_input, sizes, trials=5):
    rows = []
    for n in sizes:
        ts = []
        for _ in range(trials):
            data = make_input(n)          # build OUTSIDE the timed region
            t0 = time.perf_counter()
            fn(data)
            ts.append(time.perf_counter() - t0)
        rows.append((n, statistics.median(ts)))
    return rows
```

5.1 Four ways to get this wrong

- **Timing input construction.** Generating a random array of a million elements can take longer than sorting it. Build it outside the timed region.

- **Using the mean.** One unlucky trial — a GC pause, a context switch, a thermal event — moves a mean and does not move a median. Use the median.
- **Too narrow a size range.** $n \log n$ and n are nearly indistinguishable over one order of magnitude. Span at least three.
- **Letting the optimiser delete your work.** If the result is unused, a compiler may remove the computation entirely. Consume the output — accumulate it, print a checksum.

Read exponents off a log-log plot: the slope of $\log(\text{time})$ against $\log(n)$ is the exponent. Slope 1 is linear, slope 2 is quadratic, and $n \log n$ shows as a slope slightly above 1 that drifts upward very slowly — which is why a wide range matters.

6 The growth hierarchy, with numbers

Growth	n for ~1 second	Where you meet it
$O(1)$	Any	Hash lookup, array index.
$O(\log n)$	Any	Binary search, balanced tree operations.
$O(n)$	$\sim 10^8$	Linear scan, counting sort, BFS/DFS.
$O(n \log n)$	$\sim 10^7$	Comparison sorting, most divide and conquer.
$O(n^2)$	$\sim 10^4$	Nested loops, naive all-pairs, bubble sort.
$O(n^3)$	$\sim 10^3$	Floyd–Warshall, naive matrix multiply.
$O(2^n)$	~ 25	Subset enumeration, brute-force search.
$O(2^n \cdot n^2)$	~ 20	Held–Karp TSP over subsets.
$O(n!)$	~ 11	Permutation enumeration.

Order-of-magnitude figures for a modern machine, assuming modest constants. They are not precise and do not need to be — their value is in letting you reject an approach in ten seconds.

THE MOST USEFUL TABLE IN THE COURSE

If n is 10^6 and your idea is $O(n^2)$, it will not work. You now know this before writing any code, and you can spend the time looking for an $O(n \log n)$ formulation instead.

Most of the practical value of complexity analysis is exactly this: cheap, early rejection of designs that cannot possibly meet the budget.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J Lecture 01 — Course Overview, Interval Scheduling	Sets the level and the style of argument the course expects.
MIT 6.006 Lecture 01 — Algorithms and Computation	Gentler entry point covering models and asymptotic notation carefully. Start here if the notation is not yet automatic.
Jeff Erickson, Algorithms — Chapter 0 and the Introduction	Unusually honest about what analysis is for. Read the introduction even if you skip the chapter.
Latency Numbers Every Programmer Should Know	The constants the RAM model hides, in one page. Worth memorising the orders of magnitude.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build the benchmarking harness from §5 and verify it on a function with known complexity — a triple-nested loop is $\Theta(n^3)$. Confirm your log-log fit recovers a slope near 3.
- 2 Prove from the definitions that $2n^2 + 3n + 7 = \Theta(n^2)$. State explicit constants c_1 , c_2 , and n_0 .
- 3 Find a counterexample to the claim 'if $f = O(g)$ then $2^f = O(2^g)$ '. Explain why exponentiation breaks the relation.
- 4 Implement linear search and binary search over a sorted array. Find the n below which linear search is faster, and explain the result in terms of cache lines and branch prediction rather than operation counts.
- 5 Implement array traversal in sequential order and in a random permutation. Both are $\Theta(n)$. Measure both across sizes spanning L1, L2, L3, and main memory, and plot the ratio. This is the single most instructive measurement in the module.
- 6 Write down the growth-hierarchy table from §6 from memory. Then for three problems from your own work, state n and the complexity you can afford.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is a model of computation, and what specifically does the RAM model choose to ignore?
- 2 State the definition of O precisely, including both quantifiers. What does 'binary search is $O(n^3)$ ' mean, and why is saying it unhelpful?

- 3 Give the difference between average-case and expected-case analysis, and explain why the latter is a stronger guarantee.
- 4 Amortised analysis involves no probability. What kind of statement is it, then?
- 5 Name three effects that cause measured running time to diverge from asymptotic prediction, and give the rough magnitude of each.
- 6 Roughly what input size can an $O(n^2)$ algorithm handle in a second? An $O(2^n)$ one? Why are these numbers worth memorising?